MyLangVM — Technical Specification & Architecture Manual

A lightweight, high-performance **stack-based bytecode virtual machine** and toolchain written from scratch in C.

---

1. Core Concepts & Background

What is a Language Virtual Machine?

A **Language Virtual Machine** (Process VM) is an execution engine that emulates a virtual CPU inside software. Unlike System VMs (like QEMU or VirtualBox) which emulate hardware to run full operating systems, a Language VM executes compiled bytecode for a specific programming language runtime. Famous examples include Java's JVM, JavaScript's V8, and Python's CPython.

What is Bytecode?

Bytecode is a compact, platform-independent intermediate representation (IR) of program instructions. Source code is compiled into raw numeric opcodes and operands, which the virtual machine reads and executes sequentially using native C execution logic.

Stack-Based vs. Register-Based VM Architecture

Feature	Stack-Based VM (MyLangVM)	Register-Based VM (e.g. Lua VM)
Operand Access	Pushed/popped on Evaluation Stack	Stored in explicit registers (R0, R1...)
Instruction Size	Compact (opcodes don't store register IDs)	Larger (encodes source & destination registers)
Compiler Backend	Simple and clean to emit bytecode for	Requires complex register allocation algorithms
Addition Example	`PSH 5, PSH 10, ADD`	`ADD R1, R2, R3`

2. Architecture & Dual-Stack Memory Model

MyLangVM separates data computation from control flow through a clean dual-stack runtime architecture and a dedicated 1024-word memory array:

- **Evaluation Stack (operandStack)**: Fixed-size evaluation stack (capacity 100) holding intermediate operands, arithmetic calculation results, and comparison booleans.
- **Call Stack (callStack)**: Subroutine stack storing raw return program counter (PC) addresses during function calls (CALL / RET).
- **RAM Memory Array (memory[1024])**: A 1024-word array accessible via STORE <index> and LOAD <index> opcodes.
- **Logical Instruction Mapping**: Maps 0-based logical instruction indices to raw array offsets via instructionMapArray, allowing jump instructions (JMP, JZ, JNZ, CALL) to target instruction numbers independently of 1-word or 2-word opcode sizes.

3. Modular Project Directory Structure

```
MyLangVM/
■■■ Makefile # Root Makefile
■■■ common/ # Shared Headers Across Modules
■ ■■■ instruction.h # Shared Master Enum Opcodes (27 opcodes)
■■■ vm/ # Virtual Machine Sub-Project
■ ■■■ Makefile # Build script for VM binary (mylangvm)
■ ■■■ include/ # VM Headers (vm.h, stack.h, instruction_handlers.h)
■ ■■■ src/ # VM Source (vm.c, stack.c, instruction_handlers.c, main.c)
■■■ assembler/ # Assembler Toolchain Module [COMING SOON]
■■■ Makefile # Build script for Assembler
■■■ include/ # Assembler Headers (In Development)
■■■ src/ # Assembler Source (In Development)
```

4. Instruction Set Architecture (ISA Reference)

MyLangVM defines **27 discrete opcodes**:

Opcode	Words	Stack Effect	Description
`PSH <val>`	2	[] -> [val]	Pushes immediate integer value onto operandStack
`POP`	1	[val] -> []	Pops top element from operandStack
`DUP`	1	[val] -> [val, val]	Duplicates top element of operandStack
`SWP`	1	[a, b] -> [b, a]	Swaps top two elements of operandStack
`ADD`	1	[a, b] -> [a + b]	Pops b and a, pushes a + b
`SUB`	1	[a, b] -> [a - b]	Pops b and a, pushes a - b
`MUL`	1	[a, b] -> [a * b]	Pops b and a, pushes a * b
`DIV`	1	[a, b] -> [a / b]	Pops b and a, pushes a / b (exit on div by 0)
`MOD`	1	[a, b] -> [a % b]	Pops b and a, pushes a % b
`NEG / POS`	1	[val] -> [-val]	Flips sign of positive/negative stack top
`GT / GE / EQ`	1	[a, b] -> [0/1]	Comparison operators (Greater, Greater/Equal, Equal)
`NE / LT / LE`	1	[a, b] -> [0/1]	Comparison operators (Not Equal, Less, Less/Equal)
`JMP <target>`	2	[] -> []	Unconditional jump to logical instruction target
`JZ <target>`	2	[cond] -> []	Pops cond; jumps to target if cond == 0
`JNZ <target>`	2	[cond] -> []	Pops cond; jumps to target if cond != 0
`CALL <target>`	2	[] -> []	Pushes return address to callStack; jumps to target
`RET`	1	[] -> []	Pops return address from callStack & restores PC
`STORE <idx>`	2	[val] -> []	Pops top value and stores into memory[idx]
`LOAD <idx>`	2	[] -> [val]	Pushes memory[idx] value onto operandStack
`INPT`	1	[] -> [input]	Prompts user for integer input via stdin and pushes it

`PRNT`	1	[val] -> []	Pops top value from operandStack and prints to stdout
`HLT`	1	[] -> []	Halts program execution cleanly

5. Assembler Toolchain [COMING SOON]

An **Assembler Module** is currently under active development in the assembler/ directory. The assembler will parse human-readable assembly text files (.mylang), resolve label addresses, and emit binary bytecode files to be executed directly by **MyLangVM**.

6. Build & Run Guide

Building the VM

1. Open terminal and navigate to the vm/ directory:
cd vm

2. Run make to compile:
make

Executing Programs

• On macOS / Linux: ./mylangvm
• On Windows (MinGW): .\mylangvm.exe

Cleaning Build Artifacts

Run make clean inside vm/ to remove binaries.